

提高级 CSP-S 第 11 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 请选出以下最大的数()。
A. $(2333)_{10}$ B. $(999)_{16}$ C. $(100011111111)_2$ D. 2^{11}
- 微型计算机中,()的存取速度最快。
A. 高速缓存 B. 寄存器 C. 外存储器 D. 内存储器
- 128 KB 包含()个二进制位。
A. 128×1000 B. $128 \times 1000 \times 8$ C. 128×1024 D. $128 \times 1024 \times 8$
- 有一个空队列 Q,对下列待进队的数据元素序列 a,b,c,d,e,f 依次进行:进队,进队,进队,出队,进队,出队,出队,进队。则操作完成后队首元素为()。
A. a B. c C. d D. e
- 设变量 x 为 int 型且已赋值,则以下语句能将 x 二进制下第 0~k-1 位清零,并保持其余位不变的操作是()。
A. $x \&=(1<<k)-1$ B. $x |= (1<<k)-1$
C. $x \&=\sim((1<<k)-1)$ D. $x |= \sim((1<<k)-1)$
- 二分图是指能将顶点划分成两个部分,每一部分内的顶点间没有边相连的简单无向图。那么,有 19 条边的二分图至少有()个顶点。
A. 8 B. 9 C. 10 D. 11
- 字符串 abcde 中有()个非空子串。
A. 15 B. 14 C. 17 D. 16
- 以下排序算法中,()是不稳定排序。
A. 桶排序 B. 归并排序 C. 冒泡排序 D. 选择排序
- 由数字 1,2,3,4,5,6,7(同一数字可以重复使用)所组成的不同的三位数中有()个是 3 的倍数。
A. 43 B. 72 C. 78 D. 115
- 快速排序的最坏时间复杂度是()。
A. $O(n^2)$ B. $O(n \log^2 n)$ C. $O(n \log n)$ D. $O(n)$
- 图灵是()人。
A. 中国 B. 日本 C. 英国 D. 美国
- 用邻接矩阵来存储一张具有 n 个顶点,m 条边的图。则进行深度优先遍历运算的时间复杂度为()。
A. $O(n^2)$ B. $O(m^2)$ C. $O(nm)$ D. $O(n+m)$
- 1414 至多可以表示为()个不同的正整数之和。
A. 50 B. 51 C. 52 D. 53
- 下列语言中属于解释执行的语言是()。
A. C++ B. Pascal C. C D. Python
- 在 NOI 系列赛事中参赛选手必须使用由承办单位统一提供的设备。下列物品中不允许选手自带的是()。
A. 笔 B. 纸 C. 准考证 D. 身份证

二、阅读程序(除特殊说明外,判断题每题 1.5 分,单选题每题 3 分,共计 40 分;判断题正确填√,错误填×)

1.

```
01 #include<bits/stdc++.h>
02 using namespace std;
```

```

03  int n,a[5000],c[65536];
04  int main() {
05      cin>>n;
06
07      for (int i=0;i<n;i++) {
08          cin>>a[i];
09      }
10
11      for (int i=0;i<n;i++) {
12          for (int j=0;j<n;j++) {
13              if (i!=j) {
14                  c[a[i]^a[j]]++;
15              }
16          }
17      }
18
19      int t=0;
20
21      for (int i=0;i<=65535;i++) {
22          if (c[i]>c[t]) {
23              t=i;
24          }
25      }
26
27      cout<<t<<endl;
28      return 0;
29  }

```

除特殊声明外,假设输入的 n 和 $a[i]$ 都是不超过 65535 的非负整数,完成下面的判断题和单选题:

●判断题

- (1) n 必须小于 5000,否则程序一定会发生运行错误。()
- (2) 如果数据范围不会导致数组越界,则输出的一定是一个正整数。()
- (3) 如果输入的 $a[i]$ 不在 0~65535 范围内,则程序一定会运行错误。()
- (4) 如果数据范围不会导致数组越界,把第 21 行的“ $i=0$ ”改成“ $i=1$ ”,程序输出一定不会改变。()

●单选题

- (5) 如果 $n=10^3$,所有 $a[i]=i$,则输出结果为()。
A. 0 B. 1 C. 2 D. 1000
- (6) (4 分) 如果输入的所有数字都小于等于 15,则有()种可能的输出结果。
A. 8 B. 15 C. 16 D. 32

2.

```

01  #include<bits/stdc++.h>
02  #define N 100000
03  using namespace std;
04  int n,Fac[N+5],FacInv[N+5],MOD=1e9+7;
05  char s1[N+5],s2[N+5];

```

```

06 int C(int x,int y) {
07     return 1LL*Fac[x]*FacInv[y]%MOD*FacInv[x-y]%MOD;
08 }
09 int quick_pow(int x,int y) {
10     int t=1;
11
12     while (y) {
13         if (y & 1) {
14             t=1LL*t*x%MOD;
15         }
16
17         x=1LL*x*x%MOD;
18         y>>=1;
19     }
20
21     return t;
22 }
23 int main() {
24     Fac[0]=1;
25
26     int T,Mx=0;
27     scanf("%d",&T);
28
29     while (T--) {
30         scanf("%d",&n);
31         int t1=0,t2=0,ans=0;
32         scanf("%s",s1);
33
34         while (Mx<n) {
35             Mx++;
36             Fac[Mx]=1LL*Fac[Mx-1]*Mx%MOD;
37             FacInv[Mx]=quick_pow(Fac[Mx],MOD-2);
38         }
39
40         for (int i=0;i<n;i++) {
41             if (s1[i]=='1') {
42                 t1++;
43             }
44         }
45
46         scanf("%s",s2);
47
48         for (int i=0;i<n;i++) {
49             if (s2[i]=='1') {
50                 t2++;
51             }

```

```

52     }
53
54     int i=0;
55
56     while (t1+t2-i>n) {
57         i++;
58     }
59
60     for (;i<=min(t1,t2);i++) {
61         ans+=C(n,t1+t2-2*i),ans%=MOD;
62     }
63
64     printf("%d\n",ans);
65 }
66
67 return 0;
68 }

```

除特殊声明外,假设输入的 n 是不超过 10^5 的正整数,输入的字符串是 01 串,完成下面的判断题和单选题:

● 判断题

- (1) 每次运行到第 06 行,一定满足 $x \geq y \geq 0$ 。()
- (2) 运行到第 58 行时,一定满足 $i = t_1 + t_2 - n$ 。()
- (3) 交换第 41 行和第 49 行的操作,答案一定不会改变。()
- (4) s_1 和 s_2 实际上可以用一个字符数组来存储。()

● 单选题

- (5) 如果 $T=1, n=7$, s_1 和 s_2 分别为 0101010 和 1101101,则输出结果为()。
A. 49 B. 56 C. 63 D. 81
- (6) (4 分) 假设 $T=1$, 定义 N 为 n 的最大值, N 和 $|s_1|$ 与 $|s_2|$ 视为同阶, $\log N$ 和 $\log \text{MOD}$ 视为同阶,则这份代码的时间复杂度为()。
A. $O(N^2)$ B. $O(N \log^2 N)$
C. $O(N \log N)$ D. $O(N)$

3.

```

01 #include<bits/stdc++.h>
02 #define mod 1926
03 using namespace std;
04 int a[2001][2001],pw[1005];
05 int DP(int n,int m) {
06     if (a[n][m] !=0) {
07         return a[n][m];
08     }
09
10     if (n<m) {
11         return 0;
12     }
13
14     if (n==m || m==0) {

```

```

15         return a[n][m]=1;
16     }
17
18     if (m==1) {
19         a[n][m]=n%mod;
20         return n%mod;
21     } else {
22         a[n][m] = (DP(n-1,m-1)%mod+DP(n-1,m)%mod)%mod;
23         return a[n][m];
24     }
25 }
26 int q,n,m,ans;
27 int main() {
28     cin>>q;
29     pw[0]=1;
30
31     for (int i=1;i<=1005;i++)
32         pw[i]=2LL*pw[i-1]%mod;
33
34     while (q--) {
35         cin>>n>>m;
36         ans=(pw[m+1]-2-m+mod)%mod;
37
38         for (int j=n+1;j<=m;j++) {
39             ans--;
40
41             for (int i=j+1;i<=m;i++)
42                 ans=(ans-DP(i,j)+mod)%mod;
43         }
44
45         cout<<ans<<'\n';
46     }
47
48     return 0;
49 }

```

除特殊声明外,假设输入的 n, m 是不超过 2000 的正整数,完成下面的判断题和单选题:

● 判断题

- (1) 程序如果不使用 a 数组进行记忆化,时间复杂度仍然正确。()
- (2) 输入数据 $1 \leq q, n, m \leq 2000$, 该程序运行时一定会发生错误。()
- (3) 将程序 38 行的“ $j=n+1$ ”改成“ $j=n$ ”,程序的输出一定不会改变。()
- (4) 将程序 41 行的“ $i=j+1$ ”改成“ $i=j$ ”,程序的输出一定不会改变。()

● 单选题

- (5) (4 分) 如果 $q=1, n=3, m=5$, 则输出结果为()。
 A. 30 B. 39 C. 50 D. 59
- (6) (4 分) 如果 $q=1, n=15, m=20$, 则输出结果为()。
 A. 1799 B. 604 C. 710 D. 1794

三、完善程序(单选题,每小题 3 分,共计 30 分)

1. (方案统计) 给定一个 $n * m$ 的矩形迷宫, 有 k 种不同的颜料, 有些格子已经填上了某种颜色, 现在需要将其他格子也填上颜色, 使得从左上角到右下角的任意路径经过的格子中, 每种颜色都至多出现一次。移动只能沿着相邻的格子, 且只能向下或者向右。

请你计算所有可能的方案数量, 答案对 10^9+7 取模。

数据范围: $1 \leq n, m \leq 1000, 1 \leq k \leq 10$

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 const int mod=1000000007;
04 int n,m,k,cnt[50],a[1005][1005],sum,f[1005][1005],ps,ans,vv;
05 int dfs(int x,int y) {
06     if (y==m+1) {
07         return dfs(x+1,1);
08     }
09
10     if (x==n+1)
11         return 1;
12
13     int S=n+m,num=0,mar=0,res=0,las=0;
14     f[x][y]= ① ;
15
16     for (int i=1;i<=k;i++) {
17         if (!(f[x][y] & (1<<i-1))) {
18             ++num;
19         }
20     }
21
22     if (num< ② )
23         return 0;
24
25     if (a[x][y]==0) {
26         for (int i=1;i<=k;i++) {
27             if ( ③ ) {
28                 if (cnt[i]==0) {
29                     if (mar)
30                         res+= ④ ,res%=mod;
31                     else {
32                         mar=1;
33                         cnt[i]++;
34                         f[x][y] |=1<<i-1;
35                         las=dfs(x,y+1);
36                         f[x][y] ^=1<<i-1;
37                         cnt[i]--;
38                         res+=las;
39                         res%=mod;
40                     }

```

```

41
42             continue;
43         }
44
45         cnt[i]++;
46         f[x][y] |= 1<<i-1;
47         res+=dfs(x,y+1);
48         f[x][y] ^= 1<<i-1;
49         cnt[i]--;
50         res%=mod;
51     }
52 }
53 } else {
54     if (!(f[x][y] & (1<<a[x][y]-1))) {
55         f[x][y] |= 1<<a[x][y]-1;
56         res+=dfs(x,y+1);
57         f[x][y] ^= 1<<a[x][y]-1;
58         res%=mod;
59     }
60 }
61
62 return res;
63 }
64 int main() {
65     cin>>n>>m>>k;
66     vv= ⑤;
67
68     if (k<vv) {
69         puts("0");
70         return 0;
71     }
72
73     for (int i=1;i<=n;i++) {
74         for (int j=1;j<=m;j++) {
75             cin>>a[i][j];
76             cnt[a[i][j]]++;
77         }
78     }
79
80     ans=dfs(1,1);
81     cout<<ans<<endl;
82     return 0;
83 }

```

(1)①处应填()。

- | | |
|-------------------------------|-----------------------------|
| A. $f[x+1][y] \mid f[x][y+1]$ | B. $f[x+1][y] \& f[x][y+1]$ |
| C. $f[x-1][y] \mid f[x][y-1]$ | D. $f[x-1][y] \& f[x][y-1]$ |

(2)②处应填()。

A. $x+y$

B. $x+y+1$

C. $n+m-x-y$

D. $n+m-x-y+1$

(3)③处应填()。

A. $!(f[x][y] \ \& \ (1<i))$

B. $!(f[x][y] \ \& \ (1<i-1))$

C. $f[x][y] \ \& \ (1<i)$

D. $f[x][y] \ \& \ (1<i-1)$

(4)④处应填()。

A. res

B. num

C. las

D. S

(5)⑤处应填()。

A. $n+m-1$

B. $n+m$

C. $(n+m) * 2$

D. $(n+m) * 2-1$

2. (树的权值) 给定一棵以 1 为根节点的拥有 N 个节点的带点权树, 初始时每个点的点权均为 0, 有 Q 次操作, 每次操作有两种类型:

- 1 u , 以 u 为根节点的子树内每个点的点权均加 1。
- 2 $u \ v$, 将节点 u 到节点 v 的简单路径上的点的点权均加 1。

每次操作后询问使 $\sum_{y=1}^N A_y \times \text{dist}(x, y)$ 最小的 x , 其中 $\text{dist}(x, y)$ 表示点 x 到点 y 的路径上的边数, 如果有多个, 请输出离根节点 1 最近的那个。

数据范围: $N, Q \leq 10^5$ 。

提示: 显然答案就是求带权重心, 直接上轻重链剖分, 以线段树上二分求出中位数, 每次倍增向上跳, 判断是否符合条件即可。

```
001 #include<bits/stdc++.h>
002 #define LL long long
003 using namespace std;
004 const int N=1e5+10;
005 int n,m,fir[N],nxt[N<<1],son[N<<1],tot;
006 int dep[N],sz[N],mx[N],dfn[N],top[N],bk[N],cnt,f[N][21];
007 inline void Add(int x,int y) {
008     nxt[++tot]=fir[x];
009     fir[x]=tot;
010     son[tot]=y;
011 }
012 inline void Dfs1(int x,int fa) {
013     int i;
014     f[x][0]=fa;
015     sz[x]=1;
016     dep[x]=dep[fa]+1;
017     for (i=fir[x];i;i=nxt[i])
018         if (son[i]!=fa) {
019             Dfs1(son[i],x);
020             sz[x]+=sz[son[i]];
021         }
022     if (①)
023         mx[x]=son[i];
024 }
025 }
026 }
027 inline void Dfs2(int x,int Top) {
```



```

028     top[x]=Top;
029     dfn[x]=++cnt;
030     bk[cnt]=x;
031
032     if (mx[x])
033         Dfs2(mx[x],Top);
034
035     int i;
036
037     for (i=fir[x];i;i=nxt[i])
038         if (son[i]!=f[x][0] && son[i]!=mx[x]) {
039             Dfs2(son[i],son[i]);
040         }
041 }
042 LL T[N<<2],tg[N<<2];
043 inline void pushup(int x) {
044     T[x]=T[x<<1]+T[x<<1|1];
045 }
046 inline void pushdown(int x,int l,int r) {
047     if (tg[x]) {
048         int mid=l+r>>1;
049         T[x<<1]+=tg[x]*(mid-l+1);
050         T[x<<1|1]+=tg[x]*(r-mid);
051         tg[x<<1]+=tg[x];
052         tg[x<<1|1]+=tg[x];
053         tg[x]=0;
054     }
055 }
056 inline void Modify(int L,int R,int v,int x=1,int l=1,int r=n) {
057     if (L<=l && r<=R) {
058         T[x]+=② ;
059         tg[x]+=v;
060         return;
061     }
062
063     pushdown(x,l,r);
064     int mid=l+r>>1;
065
066     if (L<=mid)
067         Modify(L,R,v,x<<1,l,mid);
068
069     if (R>mid)
070         Modify(L,R,v,x<<1|1,mid+1,r);
071
072     pushup(x);
073 }

```

```
074 inline LL QuerySum(int L,int R,int x=1,int l=1,int r=n) {
075     if (L<=l && r<=R)
076         return T[x];
077
078     LL S=0;
079     pushdown(x,l,r);
080     int mid=l+r>>1;
081
082     if (L<=mid)
083         S+=QuerySum(L,R,x<<1,l,mid);
084
085     if (R>mid)
086         S+=QuerySum(L,R,x<<1 | 1,mid+1,r);
087
088     return S;
089 }
090 inline int QueryG(LL k,int x=1,int l=1,int r=n) {
091     if (l==r)
092         return ③ ;
093
094     pushdown(x,l,r);
095     int mid=l+r>>1;
096
097     if (T[x<<1]>=k)
098         return QueryG(k,x<<1,l,mid);
099     else
100         return QueryG(k-T[x<<1],x<<1 | 1,mid+1,r);
101 }
102 inline void Update(int x,int y) {
103     while (top[x] !=top[y]) {
104         if (dep[top[x]]<dep[top[y]])
105             swap(x,y);
106
107         Modify(dfn[top[x]],dfn[x],1);
108         x=f[top[x]][0];
109     }
110
111     if (dfn[x]>dfn[y])
112         swap(x,y);
113
114     Modify(dfn[x],dfn[y],1);
115 }
116 inline int Query() {
117     LL k=T[1]/2+1;
118     int j,i=bk[QueryG(k)];
119 }
```

```

120     if ( ④ )
121         return i;
122
123     for (j=20;~j;j--)
124         if (f[i][j] && ⑤ )
125             i=f[i][j];
126
127     return f[i][0];
128 }
129 int main() {
130     int i,j,o,x,y;
131
132     for (cin>>n,i=1;i<n;i++) {
133         cin>>x>>y;
134         Add(x,y);
135         Add(y,x);
136     }
137
138     Dfs1(1,0);
139     Dfs2(1,1);
140
141     for (j=1;j<=20;j++)
142         for (i=1;i<=n;i++)
143             f[i][j]=f[f[i][j-1]][j-1];
144
145     for (cin>>m;m--;) {
146         cin>>o>>x;
147
148         if (o==1)
149             Modify(dfn[x],dfn[x]+sz[x]-1,1);
150         else {
151             cin>>y;
152             Update(x,y);
153         }
154
155         cout<<Query()<<endl;
156     }
157
158     return 0;
159 }

```

(1)①处应填()。

A. $mx[x] < sz[son[i]]$

B. $mx[x] < son[i]$

C. $sz[mx[x]] < sz[son[i]]$

D. $sz[x] < sz[son[i]]$

(2)②处应填()。

A. $(R-L+1)$

B. v

C. $v * (R-L+1)$

D. $v * (r-l+1)$

- (3) ③处应填()。
- A. `bk[1]` B. `l` C. `bk[k]` D. `k`
- (4) ④处应填()。
- A. `QuerySum(dfn[i], dfn[i]+sz[i]-1) >= k`
B. `QuerySum(dfn[i], dfn[i]+sz[i]-1) > k`
C. `i >= k`
D. `f[i][0] > k`
- (5) ⑤处应填()。
- A. `QuerySum(dfn[i], dfn[i]+sz[i]-1) <= k`
B. `QuerySum(dfn[i], dfn[i]+sz[i]-1) < k`
C. `QuerySum(dfn[f[i][j]], dfn[f[i][j]]+sz[f[i][j]]-1) <= k`
D. `QuerySum(dfn[f[i][j]], dfn[f[i][j]]+sz[f[i][j]]-1) < k`